

✓ Name: Soumyanil Sadhu Deep

RIN: 662110663

```
1 import numpy as np
```

✓ Problem 1

```
1 x1 = np.array([2, 3, 4, 8, 9])
2 x2 = np.array([2, -3, -4, 89])
```

```
1 f_x1 = len(x1)
2 f_x2 = len(x2)
3
4 print(f"Number of features in x1: {f_x1}")
5 print(f"Number of features in x2: {f_x2}")
```

Number of features in x1: 5
Number of features in x2: 4

```
1 l1_x1 = sum(abs(x1))
2 l1_x2 = sum(abs(x2))
3 l2_x1 = np.sqrt(sum(x1**2))
4 l2_x2 = np.sqrt(sum(x2**2))
5 linf_x1 = np.max(abs(x1))
6 linf_x2 = np.max(abs(x2))
7
8 print(f"L1 norm of x1: {l1_x1}")
9 print(f"L1 norm of x2: {l1_x2}")
10 print(f"L2 norm of x1: {l2_x1}")
11 print(f"L2 norm of x2: {l2_x2}")
12 print(f"L_inf norm of x1: {linf_x1}")
13 print(f"L_inf norm of x2: {linf_x2}")
```

L1 norm of x1: 26
L1 norm of x2: 98
L2 norm of x1: 13.19090595827292
L2 norm of x2: 89.16277250063504
L_inf norm of x1: 9
L_inf norm of x2: 89

✓ Problem 2

```
1 x_size = 1024 * 1024 * 3
2
3 print(f"The length of feature vector of x: {x_size}")
```

The length of feature vector of x: 3145728

```
1 y_size = 64 * 64 * 3
2
3 print(f"The length of feature vector of y: {y_size}")
```

The length of feature vector of y: 12288

```

1 W_size = x_size * y_size
2 b_size = y_size
3
4 print(f"The number of elements in W and b are respectively {W_size} and {b_size}.")

```

The number of elements in W and b are respectively 38654705664 and 12288.

✓ Problem 3

```

1 W = np.array([[1, -1], [2, 0]])
2 print(W)
3 W_size = np.shape(W)
4
5 norm1 = 0 # initial
6 sum_j = 0
7
8 for i in range(W_size[1]):
9     sum_j = 0 # Reset sum_j for each column
10    for j in range(W_size[0]):
11        sum_j += abs(W[j][i]) # Use absolute values for L1 norm
12    norm1 = max(norm1, sum_j)
13
14 print("L1 Norm (Max Abs Column Sum) of W:", norm1)
15
16 norm_frob = np.sqrt(np.trace(np.matmul(W.T, W)))
17
18 print("Frobenius Norm of W:", norm_frob)
19
20 # Calculate L2 norm (spectral norm) - largest singular value
21 l2_norm_W = np.linalg.svd(W)[1][0]
22 print("L2 Norm (Spectral Norm) of W:", l2_norm_W)
23
24 # Calculate L-infinity norm (maximum absolute row sum norm)
25 linf_norm_W = np.max(np.sum(np.abs(W), axis=1))
26 print("L-infinity Norm (Max Abs Row Sum) of W:", linf_norm_W)

```

```

[[ 1 -1]
 [ 2  0]]
L1 Norm (Max Abs Column Sum) of W: 3
Frobenius Norm of W: 2.449489742783178
L2 Norm (Spectral Norm) of W: 2.2882456112707374
L-infinity Norm (Max Abs Row Sum) of W: 2

```

✓ Problem 4

```

1 from sklearn import datasets
2
3 iris = datasets.load_iris()
4 iris.target_names
5
6 X = iris.data[:, :3] # only selecting first three features
7 y = iris.target
8
9 from sklearn.model_selection import train_test_split
10
11 X_train, X_test, y_train, y_test=train_test_split(
12     X,y,
13     test_size=0.40,
14     train_size=0.60,
15     random_state=123,
16     shuffle=True,
17     stratify=y)

```

```
18
19 from sklearn import svm
20
21 clf = svm.SVC()
22 clf.fit(X_train, y_train)
23
24 from sklearn.metrics import accuracy_score
25 acc = accuracy_score(y_test,clf.predict(X_test) )
26 print('accuracy score :', acc)
```

accuracy score : 0.9333333333333333

✓ Problem 5

```
1 import tensorflow as tf
2 from numpy import loadtxt
3 from keras.models import Sequential
4 from keras.layers import Dense
5 from keras import optimizers
6 from tensorflow.python.keras.optimizers import *
7 import numpy as np
8
9
10 #Load dataset
11 #split into input (X) and output (y)
12 X = np.arange(1, 101)
13 X = X*1.0
14 y = np.arange(1, 101) ** (1 / 7)
15 y = y*1.0
16
17 #define keras model
18 model = Sequential()
19
20 model.add(Dense(6,input_dim=1,activation='relu'))
21 model.add(Dense(6,activation='relu'))
22 model.add(Dense(6,activation='relu'))
23 model.add(Dense(1))
24
25 #compile the keras model
26 opt = optimizers.Adam(learning_rate=0.001)
27 mse = tf.keras.losses.MeanSquaredError(
28     reduction=tf.keras.losses.Reduction.SUM)
29 model.compile(loss=mse, optimizer=opt)
30
31
32
33 #fit the keras model on the dataset (CPU)
34 model.fit(X,y,epochs=2000,batch_size=10, verbose=0)
35 model.summary()
36
37 #make class predictions with the model
38 predictions = model.predict(X)
39
40 #summarize the first 10 cases
41 for i in range(10):
42     print('%s => %.2f (expected %.2f)' %(X[i].tolist(), predictions[i], y[i]) )
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 6)	12
dense_1 (Dense)	(None, 6)	42
dense_2 (Dense)	(None, 6)	42
dense_3 (Dense)	(None, 1)	7

Total params: 311 (1.22 KB)

Trainable params: 103 (412.00 B)

Non-trainable params: 0 (0.00 B)

Optimizer params: 208 (836.00 B)

4/4 — 0s 14ms/step

1.0 => 1.70 (expected 1.00)

2.0 => 1.70 (expected 1.10)

3.0 => 1.70 (expected 1.17)

4.0 => 1.70 (expected 1.22)

5.0 => 1.70 (expected 1.26)

6.0 => 1.70 (expected 1.29)

7.0 => 1.70 (expected 1.32)

8.0 => 1.70 (expected 1.35)

9.0 => 1.70 (expected 1.37)

10.0 => 1.70 (expected 1.39)

```
/tmp/ipykernel_2489/305787602.py:42: DeprecationWarning: Conversion of an array with ndim > 0 to a
print('%s => %.2f (expected %.2f)' % (X[i].tolist(), predictions[i], y[i]))
```

✓ Problem 6

```
1 from sklearn import datasets
2 import numpy as np
3 from keras.models import Sequential
4 from keras.layers import Dense
5 from keras import optimizers
6 from tensorflow.python.keras.optimizers import *
7 import tensorflow as tf
8
9 # Load the iris dataset
10 iris = datasets.load_iris()
11
12 # Define X (sepal length) and y (petal length)
13 X_sepal_length = iris.data[:, 0] # Sepal Length is the first feature (index 0)
14 Y_petal_length = iris.data[:, 2] # Petal Length is the third feature (index 2)
15
16 # Reshape X and Y for Keras input (if they are 1D arrays)
17 X_sepal_length = X_sepal_length.reshape(-1, 1)
18 Y_petal_length = Y_petal_length.reshape(-1, 1)
```

```
1 # Define Keras model
2 model_iris = Sequential()
3
4 model_iris.add(Dense(10, input_dim=1, activation='relu')) # Input is 1 feature (sepal length)
5 model_iris.add(Dense(10, activation='relu'))
6 model_iris.add(Dense(1)) # Output is 1 feature (petal length)
7
8 # Compile the Keras model
9 opt_iris = optimizers.Adam(learning_rate=0.001)
10 mse_iris = tf.keras.losses.MeanSquaredError()
11 model_iris.compile(loss=mse_iris, optimizer=opt_iris)
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pa
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
1 # Fit the Keras model on the dataset
2 model_iris.fit(X_sepal_length, Y_petal_length, epochs=200, batch_size=5, verbose=0)
3 model_iris.summary()
4
5 # Make predictions with the model
6 predictions_iris = model_iris.predict(X_sepal_length)
7
8 # Summarize the first 10 cases
9 print('Predictions vs. Actuals for first 10 samples:')
10 for i in range(10):
11     print(f'Input (Sepal Length): {X_sepal_length[i][0]:.2f} => Predicted (Petal Length): {pre
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 10)	20
dense_5 (Dense)	(None, 10)	110
dense_6 (Dense)	(None, 1)	11

Total params: 425 (1.66 KB)

Trainable params: 141 (564.00 B)

Non-trainable params: 0 (0.00 B)

Optimizer params: 284 (1.11 KB)

5/5  0s 10ms/step

Predictions vs. Actuals for first 10 samples:

Input (Sepal Length): 5.10 => Predicted (Petal Length): 2.46 (Actual: 1.40)
Input (Sepal Length): 4.90 => Predicted (Petal Length): 2.09 (Actual: 1.40)
Input (Sepal Length): 4.70 => Predicted (Petal Length): 1.73 (Actual: 1.30)
Input (Sepal Length): 4.60 => Predicted (Petal Length): 1.55 (Actual: 1.50)
Input (Sepal Length): 5.00 => Predicted (Petal Length): 2.28 (Actual: 1.40)
Input (Sepal Length): 5.40 => Predicted (Petal Length): 3.01 (Actual: 1.70)
Input (Sepal Length): 4.60 => Predicted (Petal Length): 1.55 (Actual: 1.40)
Input (Sepal Length): 5.00 => Predicted (Petal Length): 2.28 (Actual: 1.50)
Input (Sepal Length): 4.40 => Predicted (Petal Length): 1.29 (Actual: 1.40)